EXPERIMENT 15

Career Pathways and Future of Low-Code

AIM

To analyze career opportunities, required skills, and future trends in low-code development using the Patient Appointment Management System as a practical example.

APPLICATION CONTEXT

Building the Healthify application involves form design, data organization, reports, dashboards, workflows, role-based permissions, and external service exploration. These activities connect to several career pathways in application development.

1. Career Pathways

- Low-Code Developer: builds forms, reports, dashboards, and workflows using visual development tools.
- Zoho Creator Developer: configures Creator applications and writes Deluge scripts for validation, automation, and integrations.
- Low-Code Solution Architect: plans data models, application structure, permissions, integrations, and deployment lifecycle.
- Integration Developer: connects applications to external services through REST APIs, JSON, authentication, and error handling.
- Application Security Administrator: manages role-based access, permission profiles, security policies, and audit review.

2. Skills Demonstrated by the Project

- Form design and data modelling for patient and appointment details.
- Report creation and dashboard visualization.
- Workflow configuration and criteria-based Deluge logic.
- Role and permission configuration for doctors and receptionists.
- Basic API and serverless integration concepts.
- Testing, troubleshooting, documentation, and deployment awareness.

3. Future Trends

- AI-assisted application building and natural-language generation of forms, workflows, and dashboards.
- More integration between low-code frontends and pro-code serverless backends.
- Greater focus on identity, role-based access, privacy, and auditability.
- More reusable components and connected cloud services across business applications.

4. Relevance to the Patient Appointment Management System

The project provides a practical example of translating a real-world workflow into a digital application. It can be used to demonstrate low-code development, basic automation, analytics, access control, and awareness of integration architecture.

5. Conclusion

Low-code skills complement programming and integration skills. Combining visual development with scripting, API knowledge, testing, and security awareness creates useful pathways into application development and solution engineering.

RESULT

Thus, relevant low-code career pathways, skill requirements, and future trends were reviewed in the context of the Patient Appointment Management System.